

Unit 12: Software Development

Assignment: Assignment 1 of 2 – Design, Develop, Test and Review a software program

Name: Jayden Gamble

Group: FC1

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Contents

PROGRAM SPECIFICATION	3
Purpose/User requirements:.....	3
Problem Definition.....	4
The main steps of the development (Pseudo Code or Flowchart)	5
Alternative main steps of the development (Pseudo Code or Flowchart)	6
Chosen Design	7
Variables used.....	8
Interface Designs	9
Justification of chosen design and why alternative designs were rejected.....	12
Test Plans.....	13
Justification for chosen logic.....	17
Develop and Test the Program	18
Code from the original version of the program	18
Screenshots of the original program running.....	21
Test Results for Functionality	22
Test Results for the Validation	24
Screenshots of Errors – before and after repairs	25
Feedback from peers on the usability and quality of the program.....	25
Refinements	29
Written Report	30

PROGRAM SPECIFICATION

Title:	DCM Loaning System	Language:	Python
Purpose:	Loan calculation system	Version:	1.0
Author:	Jayden Gamble	Hardware:	Unknown?
Date:	17/04/26	Operating System:	Windows

Purpose/User requirements:

The purpose of this program is

To provide Derby Computer Museum with an easy way to calculate the multiple factors for the total loaning costs of their equipment to customers, replacing the need for manually calculating which reduces the chance of math errors, and from referring to possibly inaccurate or out of date data, as everyone will have the same dataset. Which makes the loaning process more accessible to staff, as the calculations are handled by the program itself, so those who have poor maths skills can still process a loan, and for those who may have poor handwriting, which may be difficult for others to read can use the software to do the heavy lifting for them digitally, so they can also process the loan effectively and reduce the risk of error.

These are the user requirements:

The users need to have a selection of devices displayed for them to choose from, so they know what devices are available for loaning, the program also needs to validate the users inputs to prevent errors and to make sure the user is where they need to be, instead of defaulting to what could be incorrect. It must also have a simple interface that's easy to use and understand.

These are the client requirements:

The program must output the weekly and total costs based on how many weeks a loan is requested for, it also needs to calculate the loan costs with an admin charge that can be changed if a customer is a member or not.

The program must not use lots of storage or require lots of other system resources, so it can be added to all company devices. It also must be easy to modify the program by both current and future developers, so corrections, alterations, and maintenance can be completed as needed.

The client has asked that they are kept involved during each stage of development so they can provide input if seemed necessary, especially through the initial design, program development, testing, and review.

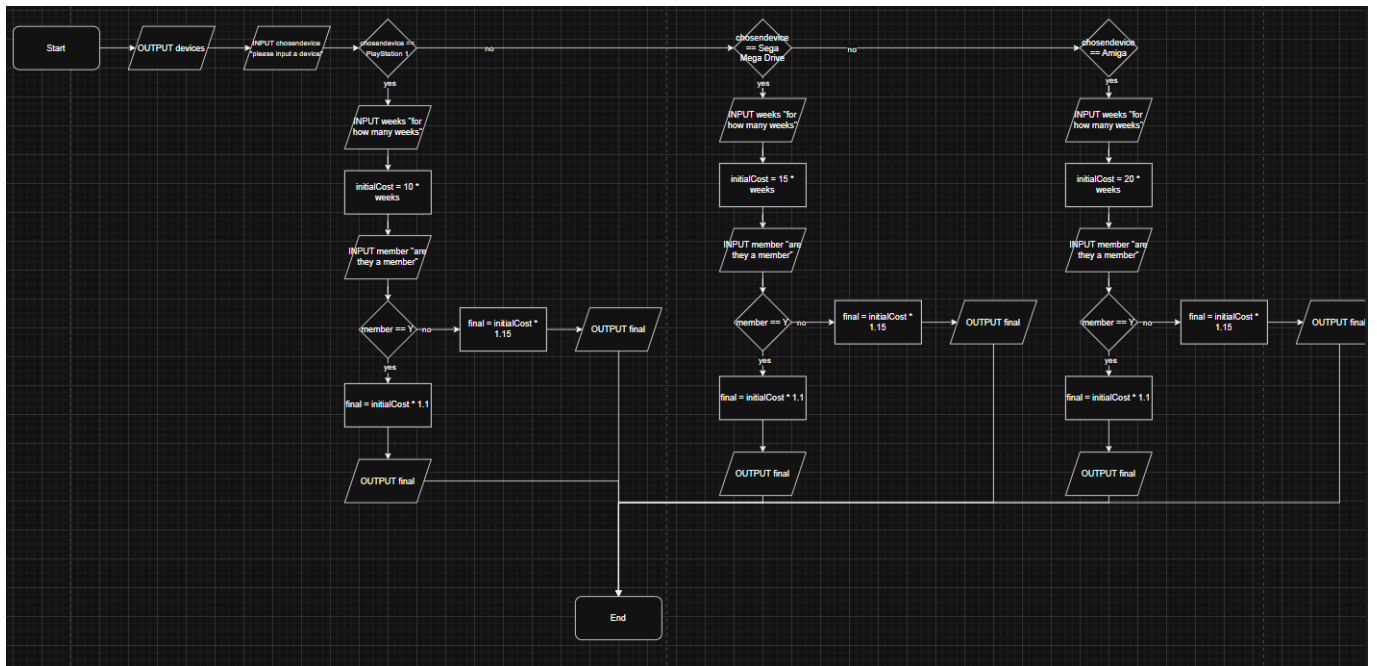
Assignment 1 of 2 – Design, Develop, Test and Review a software program

Problem Definition

The Derby College Museum is interested in loaning out their pieces of tech to enthusiasts at a cost to raise more funds for the Museum for further improvements. They want a program that handles the loaning process efficiently and easily for their staff, they likely landed on the decision of requiring a program to handle the processes because they would not need to spend extra funds on a paper-based system, which overtime could have been costly, instead with the program they would be able to reuse existing systems that are capable of running it, the program would also be able to handle the maths of the loaning process for the user, which solves the problem of some of their staff that may not be able to correctly calculate the loaning costs on their own, which without it could have caused them to give to too low or high pricing to customers, this makes the loaning process more accessible to their staff, which could improve their funds. The loaning program would also be able to be applied throughout the Museum on all of their work systems, and would be able to be used at any given time, which would help maximize the potential funds, as they would be able to start the loaning process quickly.

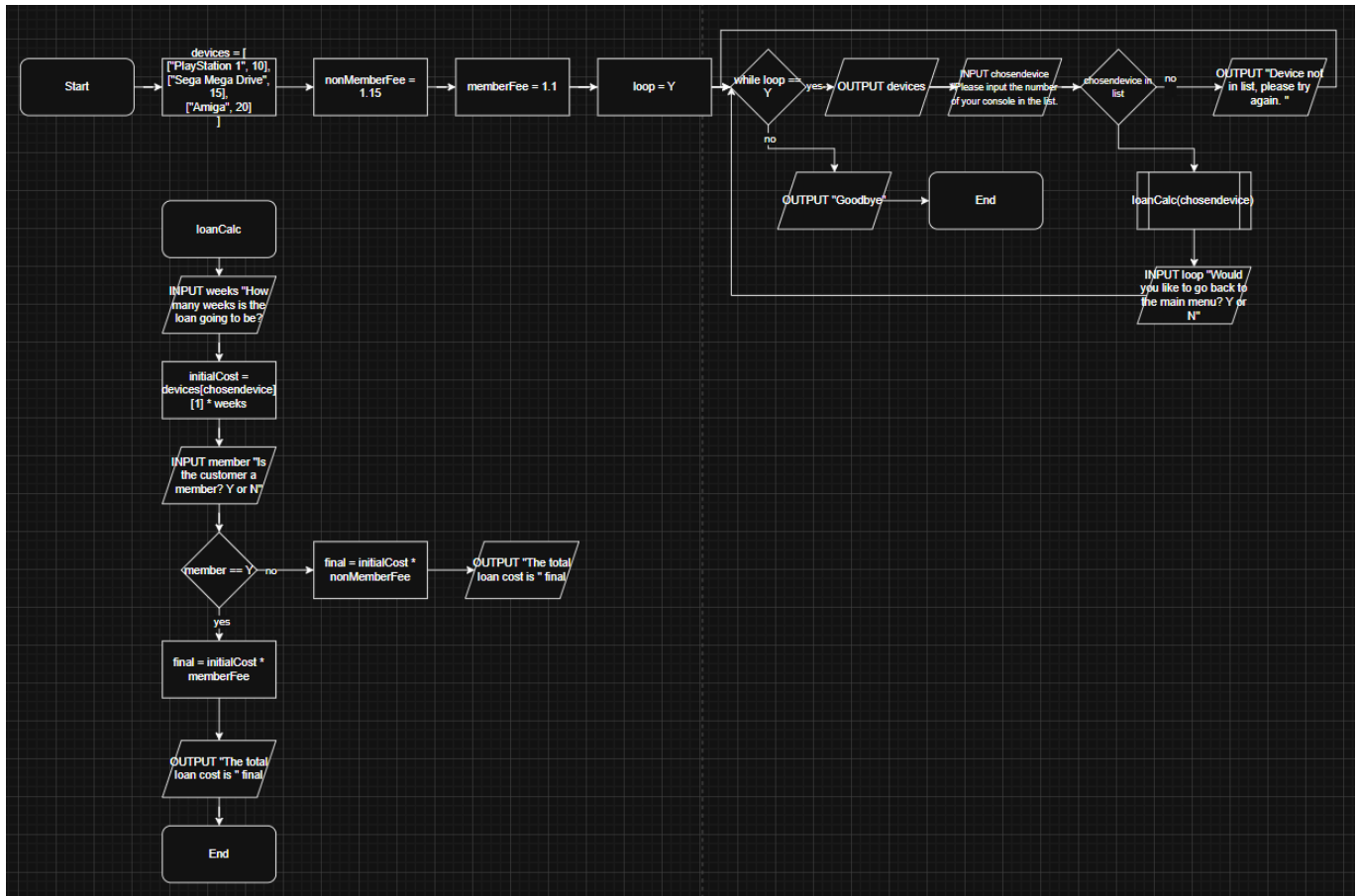
Assignment 1 of 2 – Design, Develop, Test and Review a software program

The main steps of the development (Pseudo Code or Flowchart)



Assignment 1 of 2 – Design, Develop, Test and Review a software program

Alternative main steps of the development (Pseudo Code or Flowchart)



Assignment 1 of 2 – Design, Develop, Test and Review a software program

Chosen Design

Use your chosen flow chart or pseudo code to identify the following

1. Required Inputs:

The input that the user will see at the very start of the program comes after the output of devices, which it will ask the user to input the associated number with the device, which will be used to then lead us to the next input which takes place in loan calculation asking the user to input how many weeks the loan will be, then it'll ask the user to input Y or N in regard to the customer being a member or not, and after processing completes the user will be presented to input Y or N for continuing or exiting the program.

2. Processing (Calculations/Validation):

The first validation check in our flow chart takes place after the user inputting their chosen device, which it checks if the "chosendevice" is in our list of devices, if it is, the program moves on to the loanCalc function, if it is not in our list of devices the validation will display a message and then return the user back to the start, listing the devices and asking the user to choose.

Next, the first calculation process takes place after the user inputting the amount of weeks, they want the loan to be, where it sets an initial cost from the chosen devices base price and multiplies it by the inputted amount of weeks.

Then after the user inputs Y or N for if the customer is a member, if the user inputs Y the program sets the final cost by multiplying the initial cost set previously by the member fee, similarly, if the user inputs N the program sets the final variable through calculating the value of multiplying the initial cost set earlier by the non-member fee.

3. Required Outputs:

The output the user will first see in the program is a list of the devices available for them to choose from, which links into the input that comes after which asks the user to choose from that device list, which after that, the next output which we would see if the user inputted a device not in the devices list, notifies the user that it isn't a valid option, which is essential to reduce any confusion. Later in the program within loan calculation, the user can face 2 outputs, both identical but could have a different "final" value, after the non-member calculation takes place the user is displayed a string telling the user "The total loan cost is" with the value of final added to the end, after member calculation takes place, the output is identical to non-member except for the "final" value added at the end, as it would be the final *member* cost.

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Variables used

Name	Type	Example value	Description
devices	List	[[“PS1”, 10], [“PS2”, 5]]	Stores the devices available for loaning
nonMemberFee	Integer	1.15	Stores the value used in the multiplication that takes place in loan calculation
memberFee	Integer	1.1	Same as nonMemberFee
loop	String	Y	For storing the check in our loop to continue the program
chosendevice	Integer	1	To store the users input of their chosen device so the rest of the program knows what to look for
weeks	Integer	2	To store the user inputted number of weeks that the loan will be, to be later used in calculation of the costs.
member	String	Y	To store the input of Y or N which is used to pick the correct fee charge in loan calculation.
final	Float	51.75	To store the calculated loan costs

Unit 12: Software Development

Jayden Gamble

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Interface Designs

Program Interface Design (a)

OY OOMMUSEUM

1. PlayStation 1
Base Price - £10

2. Sega Mega Drv
Base Price - £15

3. Amiga Desktop
Base Price - £20

Input the console of your choosing:

1

Thank you, how many weeks will the loan be:

4

Is the customer a member with Derby Computer Museum:

Y

The loan cost is: £44

Do you want to continue:

N

Exiting..

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Program Interface Design (b)

```
+-----+XXXXXXXXXXXXXXXXXXXXX
|  D  |  T  |  -  |  -  |  \  |  /  |
|-----|-----|-----|-----|
|  G  |  O  |  M  |  P  |  U  |  T  |
|-----|-----|-----|-----|
|  M  |  U  |  S  |  E  |  U  |  M  |
|-----|-----|-----|-----|
+-----+XXXXXXXXXXXXXXXXXXXXX

X 1. PS1          X
X   £10          X
X                X
X 2. Mega Drive  X
X   £15          X
X                X
X 3. Amiga Co..  X
X   £20          X
X                X
X                X
X                X
X                X
X                X
+-----+XXXXXXXXXXXXXXXXXXXXX
```

Please choose the desired console:

2

How many weeks does the customer want the loan to be?

12

Are they a member of Derby Computer Museum?

N

The final calculated loan totals to: £207

Do you want to restart?

Y

Returns to menu

Program Interface Design (c)

LOAN CALCULATOR

++-----+	++-----+
1 PlayStation 1 £10	2 Sega Mega Drive £15
++-----+	++-----+
++-----+	++-----+
3 Amiga Desktop £20	4 Placeholder Dev £??
++-----+	++-----+

CONSOLE >

Please input the number of your console in the list.
3

How many weeks is the loan going to be?
6

Is the customer a member? Y or N
Y

The final loan cost is: £207

Would you like to go back to the main menu? Y or N
N

Goodbye

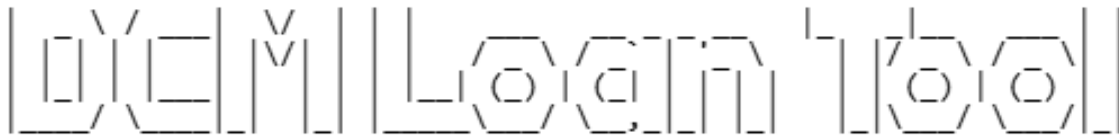
Assignment 1 of 2 – Design, Develop, Test and Review a software program

Justification of chosen design and why alternative designs were rejected

(Which design was chosen and why? Why did you reject the other designs? How will your design fulfil the stated purpose and user requirements)?

After reviewing each of the designs, I have chosen to create a fourth (D) design (see below this justification) and reject the alternatives. I made this decision as I felt the alternatives both had their own set of pros and cons, such as in Design A, which has a logo for Derby Computer Museum that isn't all that intrusive, but the rest of the design has lots of wasted space and generally is a bit "ugly", in Design B the layout design is more pleasing to look at, and there's a vertical list of devices to choose from, though this design again has lots of wasted space from the logo being too big, Design C has a pleasing design and is easy to look at, which having this is specifically mentioned in the clients requirements, though not required, the program does not show that its Derby Computer Museums program, it's just titled "Loan Calculator" which makes the program seem quite generic and uninspiring which could represent poorly among staff and customers. With all of this in mind I decided to create a better design by merging the best parts of all the designs into one, this was done so the final design best fits the client's aesthetic, and the designated purpose and requirements.

Design D:



+ + - - - - - + + +	+ + - - - - - + + +	+ + - - - - - + + +
1 PlayStation 1 £10	4 Placeholder £??	7 Placeholder £??
+ + - - - - - + + +	+ + - - - - - + + +	+ + - - - - - + + +
2 Sg Mega Drive £15	5 Placeholder £??	8 Placeholder £??
+ + - - - - - + + +	+ + - - - - - + + +	+ + - - - - - + + +
3 Amiga Desktop £20	6 Placeholder £??	9 Placeholder £??
+ + - - - - - + + +	+ + - - - - - + + +	+ + - - - - - + + +

CONSOLE >

```

Please input the number of your console in the list.
1

Please input the number of weeks the loan is going to be:
14

Is the customer a member? Please enter Y or N:
N

The total loan cost is: £161

Would you like to go back to the main menu? Y or N:
N

Goodbye
  
```

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Test Plans

Test Plan 1: Validation Testing – Use a range of Normal, Erroneous and Extreme data.

Test	Input field	Acceptable Range	Type of Data	Value Entered	Expected Result	Actual Result	Action
n	ClientAge	18 – 64	Normal	24	Program accepts value	Program accepts value	None
n			Extreme	64	Program accepts value	Program accepts value	None
n			Erroneous	65	Program rejects value	Program accepts value	Needs validation to reject >64
1	loop	Y - N	Normal	Y or N	Program accepts value		
2			Extreme	y or n	Program accepts value		
3			Erroneous	m	Program rejects value		
4	chosendevic e	1 - 3	Normal	2	Program accepts value		
5			Extreme	3	Program accepts value		
6			Erroneous	4	Program rejects value		
7	weeks	1-52	Normal	2	Program accepts value		
8			Extreme	52	Program accepts value		
9			Erroneous	53	Program rejects value		
10	member	Y - N	Normal	Y or N	Program accepts value		
11			Extreme	y or n	Program accepts value		

Assignment 1 of 2 – Design, Develop, Test and Review a software program

12			Erroneous	m	Program rejects value		
----	--	--	-----------	---	-----------------------	--	--

Leave last two columns blank until you have made the program. Then compare the expected result with the actual output and indicate an action.

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Test Plan 2: Check functionality – Complete the table with the names of input variables. Plan several different combinations of inputs and record them here. Manually calculate a possible result for each test and enter it in the Expected outcome column.

Test	chose ndevic e	weeks	member	Expected outcome	Actual outcome	Action
1	1	1	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: £11		
2	1	2	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: £23		
3	1	3	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: £33		
4	1	4	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: £46		
5	2	1	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: £16.5		

Assignment 1 of 2 – Design, Develop, Test and Review a software program

6	2	2	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 34.5		
7	2	3	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 49.5		
8	2	4	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 69		
9	3	1	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 22		
10	3	2	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 46		
11	3	3	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 66		

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Justification for chosen logic

Justification of the planned programming with regard to

- the requirements and constraints
- why the alternative design ideas were rejected
- the impact your design decisions will have on the business.

I chose the alternative design for the planned programming to best fit the client's requirements and constraints, this was because the original design was unnecessarily complex as it made no use of sub-routines, it did not make it easy to select a device as it required a precise input of the devices name, as there was no validation and does not meet the clients requirement of being able to "select" from the list of devices (like inputting a number), it also relied on using multiple hardcoded values that can't be changed easily throughout many essential areas of the code, examples of this being the cost of devices were defined and used in the definition of initialCost which is copy and pasted for every single device, which further distances from the clients requirement of making sure the program is easy to modify by current and future developers, this would have had a negative impact on the business because it would cost them more money to maintain, and could have had errors after changes from missed hardcoded values leading to possible legal troubles and loss in profits.

With all of this considered, the alternative design is the best choice, as it stores the devices in a list with its name and price at the start of the program, which makes it easy to find and edit for all developers, the non-member and member fee values used in calculations is also defined at the start of the program which makes changing the fee costs in the future much easier than in the original design. The alternate design also uses validation throughout the user's inputs to help lessen confusion, which is missing throughout the original, it also takes advantage of sub-routines to reduce complexity from copy and pasted code, which makes it less confusing to work on, as every loan calculation will be passed through the same sub-routine, instead of every device having an entirely separate but only slightly different piece of code. Finally, this design also uses outputs throughout to help understand what the program is doing, and what something is, the original design after calculation only outputs the final value, but this alternative design outputs "The total loan cost is ***final***" which makes it much more user friendly. All of this mitigates the negative impacts on the business that would have come from the original design, and the positives from the greater user experience makes the business appear more professional, attracting more customers and making it accessible to all staff.

Unit 12: Software Development

Jayden Gamble

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Develop and Test the Program

Code from the original version of the program

Includes annotations within the code showing where and why constructs or techniques have been used (e.g. local variables, sequence, iteration). Includes validation of the inputs.

```
## Imports
import time

## Devices in stock and their base cost

devices = [
    ["PlayStation 1", 10],
    ["Sg Mega Drive", 15],
    ["Amiga Desktop", 20]
]

## Fee multiplication values

## 1.15 is the equivalent of 15% added to the total cost
## 1.1 is the equivalent of 10% added to the total cost

nonMemberFee = 1.15
memberFee = 1.1

## Helpers for more appealing design where needed


def inputConsole(inputData, clear = False):
    ## clear if we need to, same method as in menu
    if clear == True:
        print('\n' * 80)

    print("CONSOLE > ")
    outputData = input(inputData)
    return outputData


def outputConsole(outputData, clear = False):
    ## clear if we need to, same method as in menu
    if clear == True:
        print('\n' * 80)

    print("CONSOLE > ")
    print(outputData)


def displayMenu():
    ## Print 80 new lines to "clear" the console
    ## actually clearing the console requires OS specific methods
    print('\n' * 80)
    print("""

|       |    |
|-------|----|
| PSX   | 10 |
| SMD   | 15 |
| AMIGA | 20 |

"""))
```

Unit 12: Software Development

Jayden Gamble

Assignment 1 of 2 – Design, Develop, Test and Review a software program

[illegible]

Assignment 1 of 2 – Design, Develop, Test and Review a software program

```
## User has given us a number, need to check if we have that device in our list
chosenDevice = int(inputConsole("Please input the number of your console in the list: "))

## Lists start with 0, but the user doesn't know that, so we take 1 away (so 1 is actually the first device).
realChosenDevice = chosenDevice - 1

try:
    ## If the input was right we'll go through the if statement, if not then we go through the exception, then we loop back to the start
    if devices[realChosenDevice]:
        ## Process calculation
        loanCalc(realChosenDevice)

        ## We finished loan calc, now to determined if the user wants to continue
        loop = input("Would you like to go back to the main menu? Y or N: ")

except:
    print("Device not in list, please try again.")
    time.sleep(2)

## User said no to loop, so we wait a few seconds for them to see the goodbye, then close
print("Goodbye")

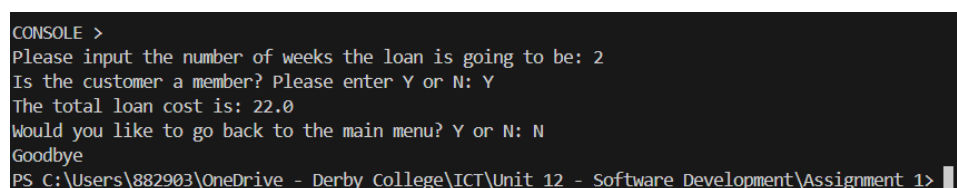
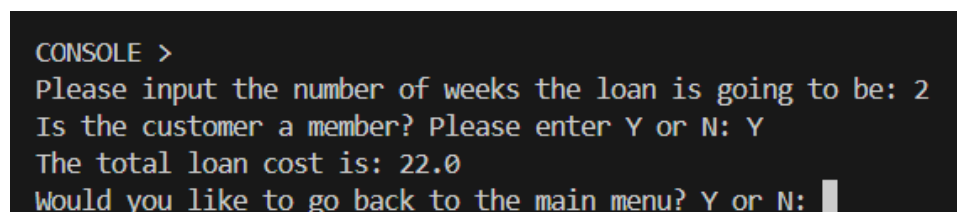
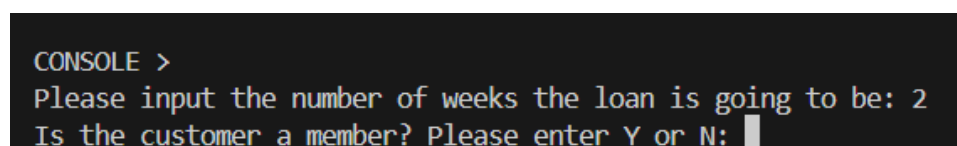
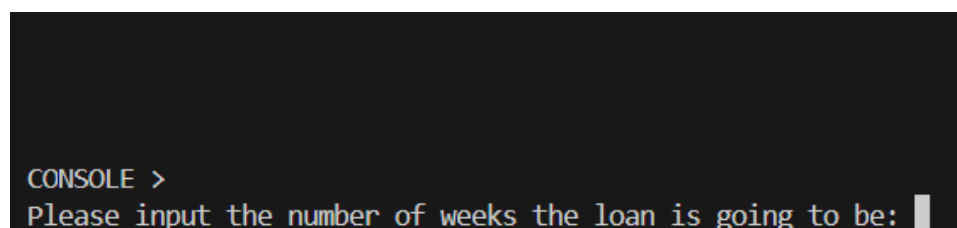
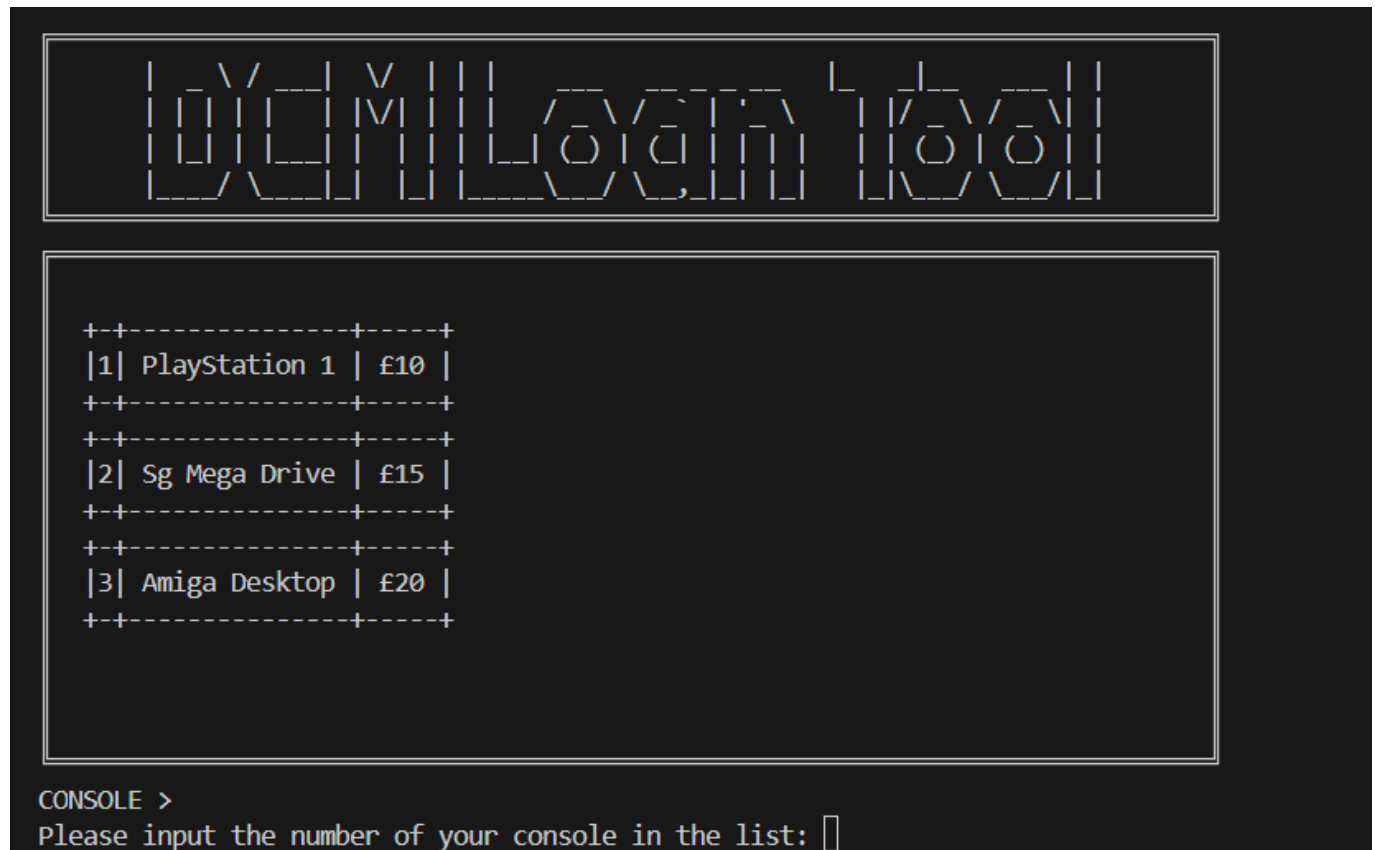
## Start
if __name__ == "__main__":
    main()
```

Unit 12: Software Development

Jayden Gamble

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Screenshots of the original program running



Assignment 1 of 2 – Design, Develop, Test and Review a software program

Test Results for Functionality

Copy the functionality test plan from above and complete it.

Test	chosedevice	weeks	member	Expected outcome	Actual outcome	Action
1	1	1	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 11	Same as expected	None
2	1	2	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 23	Same as expected	None
3	1	3	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 33	Same as expected	None
4	1	4	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 46	Same as expected	None
5	2	1	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 16.5	Same as expected	None

Assignment 1 of 2 – Design, Develop, Test and Review a software program

6	2	2	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 34.5	Same as expected	None
7	2	3	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 49.5	Mostly same as expected, except for minor bug with the total cost being "49.50000000 000001"	Fixed by rounding the total cost to 2 nd decimal place
8	2	4	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 69	Same as expected	None
9	3	1	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 22	Same as expected	None
10	3	2	N	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 46	Same as expected	None
11	3	3	Y	The program moves to ask the user how many weeks, then it will ask if the user is a member, then displays a total loan cost of: 66	Same as expected	None

Unit 12: Software Development

Jayden Gamble

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Test Results for the Validation

Copy the validation test plan from above and complete it.

Test	Input field	Acceptable Range	Type of Data	Value Entered	Expected Result	Actual Result	Action
1	loop	Y - N	Normal	Y or N	Program accepts value	Program accepts value	None
2			Extreme	y or n	Program accepts value	Program always ends loop	Changed loop to be put through upper
3			Erroneous	m	Program rejects value	Program ends loop	Added validation to ask again for the correct input
4	chosendevic e	1 - 3	Normal	2	Program accepts value	Program accepts value	None
5			Extreme	3	Program accepts value	Program accepts value	None
6			Erroneous	4	Program rejects value	Program rejects the value	None
7	weeks	1-52	Normal	2	Program accepts value	Program accepts value	None
8			Extreme	52	Program accepts value	Program accepts value	None
9			Erroneous	53	Program rejects value	Program accepts value	Added validation to reject > 52
10	member	Y - N	Normal	Y or N	Program accepts value	Program accepts value and goes the right path	None
11			Extreme	y or n	Program accepts value	Program always processes with nonMemberFee	Changed member to be put through upper
12			Erroneous	m	Program rejects value	Program always processes with nonMemberFee	Added validation to ask again for correct input

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Screenshots of Errors – before and after repairs

Before: Calculation error

```
print("The total loan cost is:", final)
```

```
CONSOLE >  
Please input the number of weeks the loan is going to be: 3  
Is the customer a member? Please enter Y or N: Y  
The total loan cost is: 49.500000000000001  
Would you like to go back to the main menu? Y or N: █
```

After: Rounding to fix calculation error

```
print("The total loan cost is:", round(final, 2))
```

```
CONSOLE >  
Please input the number of weeks the loan is going to be: 3  
Is the customer a member? Please enter Y or N: Y  
The total loan cost is: 49.5  
Would you like to go back to the main menu? Y or N: █
```

Before: Loop exits on lowercase or letter that isn't Y or N

```
CONSOLE >  
Please input the number of weeks the loan is going to be: 3  
Is the customer a member? Please enter Y or N: Y  
The total loan cost is: 49.5  
Would you like to go back to the main menu? Y or N: y  
Goodbye
```

```
Would you like to go back to the main menu? Y or N: b  
Goodbye
```

```
## We finished loan calc, now to determined if the user wants to continue  
loop = input("Would you like to go back to the main menu? Y or N: ")
```

Assignment 1 of 2 – Design, Develop, Test and Review a software program

After: Loop now reconfirms and works with lowercase

```
CONSOLE >
Please input the number of weeks the loan is going to be: 2
Is the customer a member? Please enter Y or N: Y
The total loan cost is: 22.0
Would you like to go back to the main menu? Y or N: b
Please enter only Y or N, would you like to go back to the main menu?: y
```

```
## We finished loan calc, now to determined if the user wants to continue
loop = input("Would you like to go back to the main menu? Y or N: ").upper()

## User put something other than Y or N, so we ask again until they do.
while loop != "Y" and loop != "N":
    loop = input("Please enter only Y or N, would you like to go back to the main menu?: ").upper()
```

Before: Weeks can be infinite or negative

```
## Ask the user for the total amount of weeks, also clear all the stuff from earlier
## so we can keep this calculation the focus
weeks = int(inputConsole("Please input the number of weeks the loan is going to be: ", True))
```

```
CONSOLE >
Please input the number of weeks the loan is going to be: -1
Is the customer a member? Please enter Y or N: █
```

After: Weeks are limited from 1 to 52

```
CONSOLE >
Please input the number of weeks the loan is going to be: 0
Please input a valid number of weeks (1 to 52) for how long the loan will be: 0
Please input a valid number of weeks (1 to 52) for how long the loan will be: 53
Please input a valid number of weeks (1 to 52) for how long the loan will be: 52
Is the customer a member? Please enter Y or N: █
```

```
## Ask the user for the total amount of weeks, also clear all the stuff from earlier
## so we can keep this calculation the focus
weeks = int(inputConsole("Please input the number of weeks the loan is going to be: ", True))

## User gave an amount we can't do, ask again with extra context.
while weeks > 52 or weeks < 1:
    weeks = int(input("Please input a valid number of weeks (1 to 52) for how long the loan will be: "))
```

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Before: Member always chooses N path if anything other than “Y” is inputted

```
member = input("Is the customer a member? Please enter Y or N: ")

if member == "Y":
    ## User said yes, so we use that initialCost from earlier with the memberFee
    final = initialCost * memberFee
else:
    ## User said no, so use initialCost from ealier with nonMemberFee
    final = initialCost * nonMemberFee
```

```
CONSOLE >
Please input the number of weeks the loan is going to be: 2
Is the customer a member? Please enter Y or N: y
The total loan cost is: 23.0
Would you like to go back to the main menu? Y or N: █
```

After: Member now has validation and supports lowercase inputs

```
CONSOLE >
Please input the number of weeks the loan is going to be: 2
Is the customer a member? Please enter Y or N: B
Is the customer a member? Please enter only Y or N: y
The total loan cost is: 22.0
Would you like to go back to the main menu? Y or N: █
```

```
member = input("Is the customer a member? Please enter Y or N: ").upper()

## User gave soemthing we don't understand, ask again.
while member != "Y" and member != "N":
    member = input("Is the customer a member? Please enter only Y or N: ").upper()

if member == "Y":
    ## User said yes, so we use that initialCost from earlier with the memberFee
    final = initialCost * memberFee
else:
    ## User said no, so use initialCost from ealier with nonMemberFee
    final = initialCost * nonMemberFee
```

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Feedback from peers on the usability and quality of the program

The program works well, and it's pretty robust as it wasn't easy to cause errors. The main menu is nicely laid out which made it easier to use, and its design fits the museum's theme well.

Though, when it gets to the loaning part all of the questions and outputs are all cramped together which was confusing as it wasn't immediately clear as on what to look at, which made it easy to completely overlook the final cost.

It could be a good idea to show the total cost in something that looks like a receipt or a ticket, like how the main menu looks. The program also feels too fast, which is a bit confusing as it feels like you could have done something wrong or you've missed something important, it might be worth adding some delays.

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Refinements

Refinements and improvements to the software program, taking account feedback, and the quality of the code (e.g. maintainability, portability and usability).

After listening to the feedback, I have decided to adopt the suggestions of adding delays, and a receipt style final cost within the program, as I feel it would help improve the user experience, the delays would make it easier to notice a change after executing a task, as some users might focus on their inputs on the keyboard rather than looking at the screen all the time, it also makes the program seem more reliable, as an extremely fast response could make some think that something has silently gone wrong inside the program, regarding the receipt style output, I will integrate this to the best of my abilities, clearly showing the total and weekly costs and if possible some extra information inside a basic interface resembling a receipt, and also clear old prompts when they are no longer relevant.

I will also be making some extra improvements to the program that I noticed should be implemented, such as adding the missing delay on the “Goodbye” message so the user sees the goodbye, to improve maintainability I will also replace the static base console costs in the menu to match what is in the devices list.

To see the full refined code please look at the python file attached with the assignment, I chose to not paste it in here, so it doesn't clutter the document.

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Written Report

This documents each stage of the process, with a description of any difficulties encountered and how they were resolved, including changes made to the design.

The initial stage of implementing the code for the program was started by referring to the chosen flow chart design from earlier in this assignment, initially, there was no problems and the code was matching the flow chart almost identically, however, I then realised my flow chart design did not take into consideration how the main menu would be displayed, as it only outputted the devices list, and if I was to swap the devices output in with a print of a main menu, it would make the program extremely cluttered, adding difficulty to making changes in the code for current or possible future maintainers of the program, so I decided the best option was to move parts of the programs code into sub-routines and make use of the main function, which helped make the program easier to work with.

Unfortunately, this led to me being met with multiple other problems, so I had to make even more changes that would further distance from the flow chart design. I had multiple errors and glitches that needed fixing, one example being with the loop, I was unable to change the loop value from within the loop itself, as it was defined outside of the main function, so I fixed this by moving the loop definition into it.

To create my design effectively, I had created multiple sub-routines for handling input and output, with the ability to “clear” the console, which there was no universal way to clear the console as it was OS specific, so when the value for clearing is true the program prints 80 new lines to appear as if the previous prompts were cleared. I also created a sub-routine for displaying the menu easily, which also helps maintenance.

After the creation of the initial program, I went through the testing process and discovered a few problems that required fixing, one of them being the total cost output, as there was a glitch where the cost could have sometimes have .000001 at the end, which I fixed by rounding the output number to the 2nd decimal place. Another issue I encountered from the test plan was there was validation implemented incorrectly or completely missing, which I fixed by adding upper to inputs for Y or N prompts, and correctly adding in the missing prompts.

After this I went to get user feedback to help improve the program, which from the feedback I made changes to the program to add processing delays in-between prompts, and made the outputs of the loan calculation display in a receipt style format, to help improve the UX. Which this was added without almost any issue, there was a bug where the program would display “device not in list” message even though the user had already passed that point, which this error was stemmed from an exception happening in the receipt handling where int’s were not converted to string, after doing that the program managed to display the receipt without issue.

Assignment 1 of 2 – Design, Develop, Test and Review a software program

Review of Final Program

- Using your test results explain how your program is fit for purpose and meets the original requirements, add screenshots and explain what they prove.
- Consider the outcomes of the testing and review the extent to which the program solves the given problem. Gather feedback from others and consider changes they would like to make in the light of this feedback.
- Evaluate the final software programs against the initial designs and show what changes you made to the design when you came to implement it.
- Evaluate the quality of the code, and justify any changes made, making recommendations for further improvements.

Reviewing this program, it is easy to say that it fully meets the client requirements, and it is fit for Derby Computer Museums purpose; to add credibility my claims, I will be going over the work I have completed to showcase this.

Looking at the test results (refer to page 22 to 24), it shows that I have extensively tested edge-cases and “invalid” inputs throughout the program, which this was necessary to correctly meet the client’s requirement of “There should be validation of user input to avoid either the program crashing or users not getting the setting they need.” The testing was essential as it helped point out validation that needed implementing and had also showed potential issues in the program that I had missed to my attention, which all of this in mind, it shows the test results was needed so that I could fully meet the validation client requirement, which indirectly helped to meet the requirement of “The program must have a simple interface and be very easy to use”, as the extra validation that was implemented helped improve the user experience from the guidance and validity it provides. With all of this in mind, it is clear that the program solves the client’s problem as it fits the designated purpose, which means Derby Computer Museum can begin to progress with utilizing the program to calculate their console loan costs, meaning they will be able to put more funds into museum itself.

The screenshots I have pasted below utilize the previously improperly validated variables seen in the test plans, which I have since applied fixes to correct the errors.

```
CONSOLE >
Please input the number of weeks the loan is going to be: -1|
```

```
CONSOLE >
Processing...

Please input a valid number of weeks (1 to 52) for how long the loan will be: |
```

```
CONSOLE >
Is the customer a member? Please enter Y or N: y|
```

Assignment 1 of 2 – Design, Develop, Test and Review a software program

```
=====
TOTAL: £22.0
WEEKLY: £11.0

-----
INITIAL: £20
FEES: £2.0
WEEKS: 2
=====
                THANK YOU

Would you like to go back to the main menu? Y or N: |
```

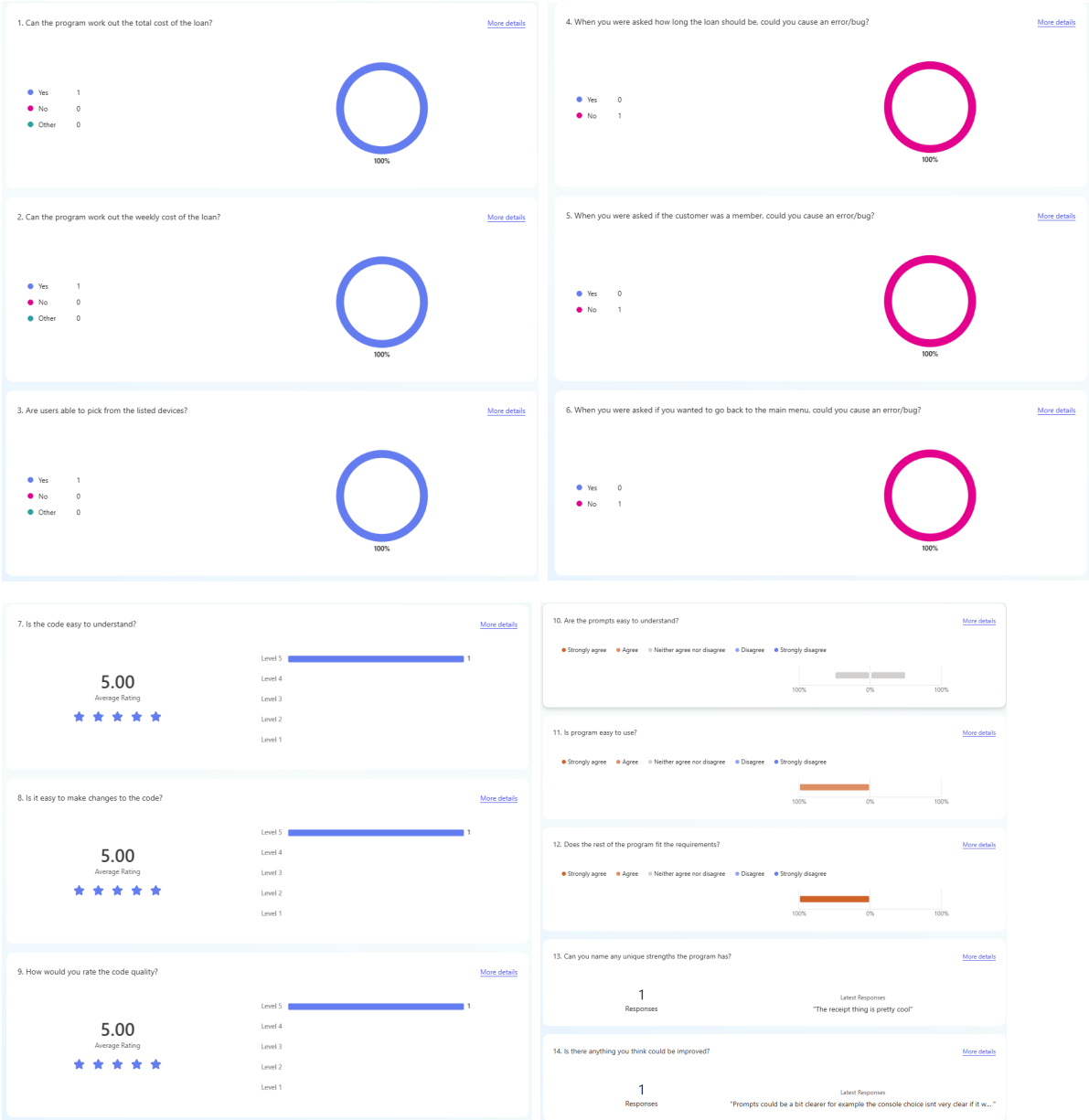
```
Would you like to go back to the main menu? Y or N: b
Please enter only Y or N, would you like to go back to the main menu?: n
Goodbye
|
```

These further prove that the test plans were critical for meeting the purpose and requirements, because it found bugs that would've made the program unsuitable for Derby Computer Museum, leading to poor UX, and it would completely fail to meet the requirement that there should be validation to avoid errors or wrong settings.

I have also created the program in a way that makes future development changes easy, which was an essential requirement for the client, I will go into further detail on this in my evaluation.

I have also created a form to gather extra feedback from my peers, which was mostly positive; they were unable to find/cause bugs or errors, they also rated the codebase highly as being easy to understand and make changes to, and that it was of good quality, finally, on the near end of form in the question asking about any unique strengths the program has they wrote about the receipt. There was also some feedback that suggested to make improvements, they put "Neither agree nor disagree" for "Are the prompts easy to understand?", which in the final question they went into detail as to why, saying that the prompts could be clearer and gave an example noting that it isn't very clear in the console choice if it is asking for the name or the index in the list.

Assignment 1 of 2 – Design, Develop, Test and Review a software program



14. Is there anything you think could be improved?		
1 Responses		
ID ↑	Name	Responses
1	Callum Low	Prompts could be a bit clearer for example the console choice isnt very clear if it wants the consoles name or the index.

Which they make a valid point, I'd take this feedback into consideration in later revisions of the program, aiming to reword the console prompt and find any other remaining poorly worded prompts to reword them too.

Finally, in this evaluation I'll be going over the final program and the differences it has against the flow-chart and the original program.

The flow chart and the final program share some similarities, the concept remains the same, but the program had a lot of changes, which makes the flow chart design not match. The flow chart

Unit 12: Software Development

Jayden Gamble

Assignment 1 of 2 – Design, Develop, Test and Review a software program

shows the intended basic functionality of the program, which was defining the loop, then entering the loop condition where it would output the list of devices and then ask for the number of it, then it would check if the device is in the list, which if it was it would then go through the loan sub-routine, which after calculation would output the total cost. Which looking at the final program we can see several multiple design changes both in code and in the user interface.

Beginning with start of the final program design, instead of just outputting the devices variable there is a visually appealing main menu, which is also separated into its own sub-routine. I did this because it improved the user experience, which the menu makes the program feel less “advanced” to new users, the reason why I separated displaying the menu into a sub-routine was to make it easier to implement any further changes that may be made in the future, such as swapping out the menu for something else, and the separation from the main part of the program reduces clutter within the core functions which will make development a much less painful process, which overall enhances the quality of the codebase.

```

41 def displayMenu():
42     ## Print 80 new lines to "clear" the console
43     ## actually clearing the console requires OS specific methods
44     print('\n' * 80)
45     print("""
46 |   _\ / _ | \ | |   _ _ _ _ _ |   _ | _ _ | |
47 | | | | | | | \ | |   / \ / \ / \ | ' \   | / \ / \ |
48 | | | | | | | | | | | ( ) | ( ) | | | |   | | ( ) | ( ) |
49 | _ / \ _ | | | | _ \ / \ _ _ | |   | | \ / \ / \ |
50
51
52
53 +-+-----+-----+
54 |1| """" + devices[0][0] + """" | £"""" + str(devices[0][1]) + """" |
55 +-+-----+-----+
56
57 |2| """" + devices[1][0] + """" | £"""" + str(devices[1][1]) + """" |
58 +-+-----+-----+
59
60 |3| """" + devices[2][0] + """" | £"""" + str(devices[2][1]) + """" |
61 +-+-----+-----+
62
63
64
65
66

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

QML Login Tool

```

+-----+-----+
|1| PlayStation 1 | £10 |
+-----+-----+
|2| Sg Mega Drive | £15 |
+-----+-----+
|3| Amiga Desktop  | £20 |
+-----+-----+

```

```
CONSOLE >
Please input the number of your console in the list:
```

Assignment 1 of 2 – Design, Develop, Test and Review a software program

```
19
20 def inputConsole(inputData, clear = False):
21     ## clear if we need to, same method as in menu
22     if clear == True:
23         print('\n' * 80)
24
25     print("CONSOLE > ")
26     outputData = input(inputData)
27
28     outputConsole("Processing...", True)
29     time.sleep(2)
30
31     return outputData
32
33 def outputConsole(outputData, clear = False):
34     ## clear if we need to, same method as in menu
35     if clear == True:
36         print('\n' * 80)
37
38     print("CONSOLE > ")
39     print(outputData)
40
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CO

CONSOLE >
Processing...

I also implemented a new set of sub-routines that handle some inputs and outputs I did this so I could easily add delays between prompts, so it felt like the program was working as intended, I also did it so I could easily create the appearance of clearing the console consistently throughout. This also being a sub-routine improves the code quality as it means I haven't copied and pasted their functionality everywhere its needed, which would clutter the code, massively lowering its quality, and increasing the time spent and difficulty in maintenance.

There are some further improvements that could be made, such as implementing the feedback improvements discussed in the final review, or importantly in future, the menu could be rewritten to adjust dynamically with items added to the list, as of right now if you add a device to the devices variable it won't be on the main menu, it has to be manually added.

Assignment 1 of 2 – Design, Develop, Test and Review a software program

BIBLIOGRAPHY